



Marco Faella

Controllo di uguaglianza tra oggetti

Lezione n. 6

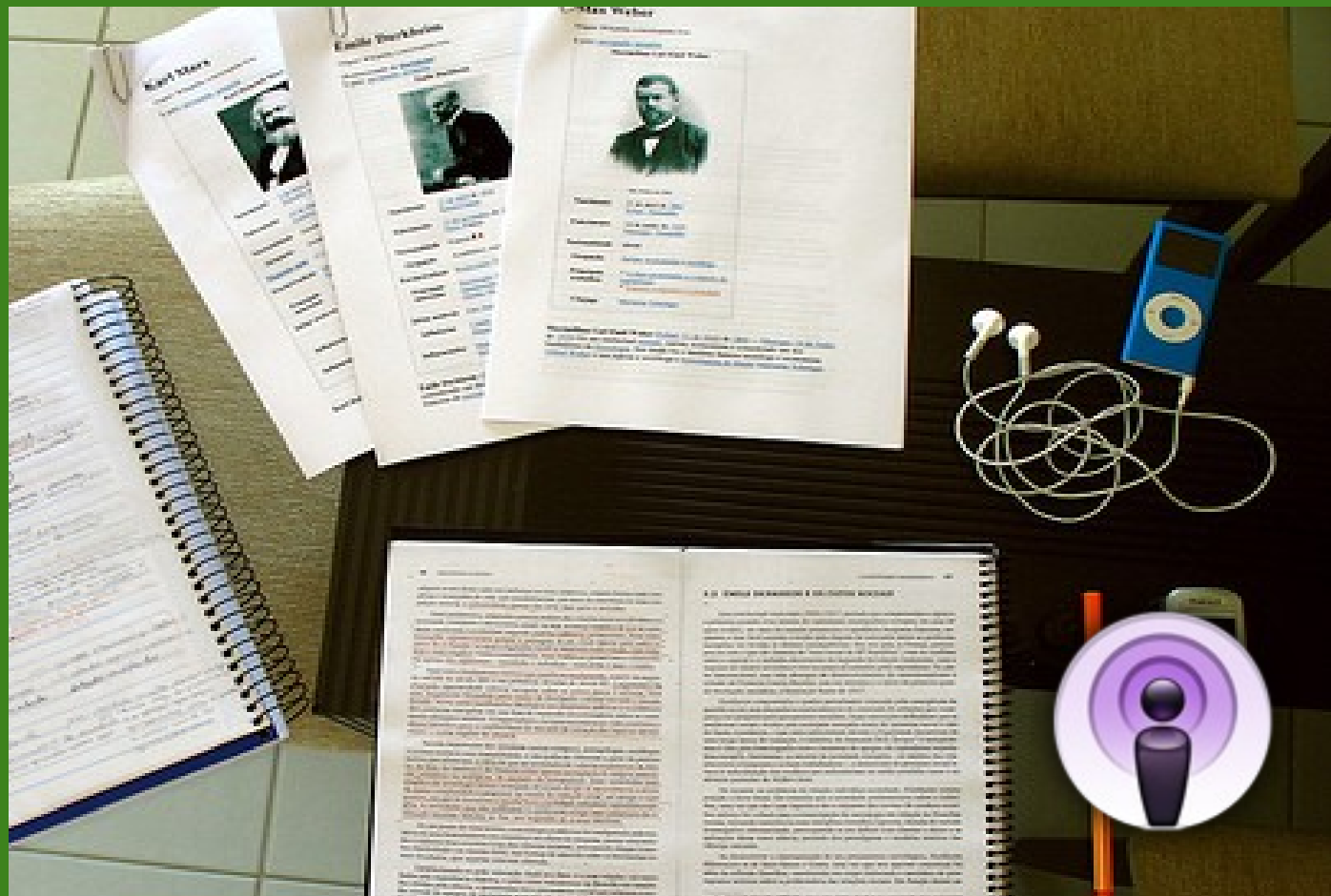
Parole chiave:
Java

Corso di Laurea:
Informatica

Insegnamento:
Linguaggi di Programmazione II

Email Docente:
faella.didattica@gmail.com

A.A. 2009-2010



Pre-requisiti: intro alla riflessione, metodo getClass, operatore .class

- Nel linguaggio Java, l'operatore == stabilisce se due riferimenti puntano al medesimo oggetto
 - Questa è la relazione di *identità*
- Spesso, è utile considerare uguali due oggetti distinti, secondo un criterio dettato di volta in volta dal contesto applicativo
- Il modo standard di confrontare oggetti è tramite il metodo equals, definito dalla classe Object

```
public boolean equals(Object x)
```

- In generale, il metodo equals prende come argomento un oggetto x e restituisce vero se questo oggetto (this) è da considerarsi "uguale" ad x, e falso altrimenti
- Nella classe Object, il metodo equals non fa altro che confrontare gli indirizzi di this e di x con ==

- Ad esempio, consideriamo un programma di gestione del personale
- La classe Employee rappresenta un impiegato ed è caratterizzata da nome (stringa), salario mensile (numero intero) e capoufficio (un altro impiegato)
- Lo schema della classe è il seguente:

```
class Employee {  
    private String name;  
    private int salary;  
    private Employee boss;  
    ...  
}
```

- Supponiamo che due impiegati vadano considerati uguali se hanno lo stesso nome e lo *stesso* capoufficio
 - ovvero, l'applicazione suppone che non ci possano essere due impiegati omonimi all'interno dello stesso ufficio
- Supponiamo che la relazione "essere capoufficio di" disponga gli impiegati in un albero, alla cui radice vi è un impiegato senza capoufficio (boss==null)

- Una possibile implementazione del metodo equals per la classe Employee è la seguente

```
public boolean equals(Object o) {  
    if (o == null) return false; // questo controllo è ridondante; perché?  
    if (!(o instanceof Employee)) return false;  
    Employee e = (Employee) o;  
    return name.equals(e.name) &&  
        (boss==e.boss || (boss!=null && boss.equals(e.boss)));  
}
```

- All'inizio, controlliamo che il riferimento passato punti effettivamente ad un Employee
- Confrontiamo i nomi con equals
- Confrontiamo anche i capiufficio con equals (sono impiegati anche loro, quindi vale per loro la stessa assunzione fatta per gli impiegati semplici)
- La forma complessa dell'espressione condizionale è dovuta al caso in cui uno dei due impiegati in questione, o entrambi, siano al vertice dell'azienda

- Il linguaggio Java richiede che qualunque ridefinizione del metodo equals rispetti le seguenti proprietà:
 - **riflessività:**
per ogni oggetto x , $x.equals(x)$ è vero
 - **simmetria:**
per ogni coppia di oggetti x ed y ,
 $x.equals(y)$ è vero se e solo se $y.equals(x)$ è vero
 - **transitività:**
per ogni terna di oggetti x , y e z ,
se $x.equals(y)$ è vero e $y.equals(z)$ è vero,
allora anche $x.equals(z)$ è vero
- Queste sono le condizioni proprie di una relazione di equivalenza

Ad esempio, valutare la **validità** delle seguenti definizioni di uguaglianza tra due Employee:

	Employee a, Employee b	Validità
(a)	stesso nome	
(b)	entrambi i salari maggiori di 1000	
(c)	il salario di a è maggiore o uguale di quello di b	
(d)	stesso nome oppure stesso salario	

Come esercizio, trovare un controesempio concreto per ogni proprietà violata da queste definizioni

Ad esempio, valutare la **validità** delle seguenti definizioni di uguaglianza tra due Employee:

	Employee a, Employee b	Validità
(a)	stesso nome	ok
(b)	entrambi i salari maggiori di 1000	non riflessiva
(c)	il salario di a è maggiore o uguale di quello di b	non simmetrica
(d)	stesso nome oppure stesso salario	non transitiva

Come esercizio, trovare un controesempio concreto per ogni proprietà violata da queste definizioni

- Bisogna prestare attenzione all'interazione tra il metodo equals e le **sottoclassi**
- Ad esempio, supponiamo che la classe *Employee* abbia una sottoclasse ***Manager***
- Gli oggetti Manager hanno un campo aggiuntivo intero chiamato "bonus"
- In base al contesto applicativo, in fase di progettazione, va deciso come si deve comportare il metodo equals con oggetti appartenenti a sottoclassi diverse
- In particolare, bisogna porsi le seguenti domande:
 - 1) Il criterio di confronto tra oggetti di una sottoclasse (ad es., due Manager) è diverso da quello che vale tra oggetti della superclasse (ad es., due Employee)?
 - 2) Un oggetto di una sottoclasse (es., Manager) può essere considerato uguale ad uno di una superclasse (es. Employee)?

Come risposta ai quesiti precedenti, sorgono due *scenari standard*:

- 1) Il criterio di confronto è **uniforme** in tutta la gerarchia e coincide con quello che si applica alla sua radice; quindi oggetti di sottoclassi diverse possono anche risultare uguali tra loro
- 2) Il criterio di confronto **cambia** nelle sottoclassi; oggetti di sottoclassi diverse sono sempre considerati diversi

- Nel **primo scenario**, in tutta la gerarchia vale il criterio stabilito per la sua radice
- Ad esempio, supponiamo che la classe Employee abbia un campo “codice fiscale”
- Come sappiamo, il codice fiscale identifica univocamente una persona
- Quindi, è ragionevole confrontare soltanto quel campo per stabilire se due Employee sono uguali
- Inoltre, lo stesso criterio si può applicare anche a due Manager, oppure per confrontare un Employee e un Manager

- L'implementazione del primo scenario è estremamente semplice:
 - La radice della gerarchia (ad es., Employee) fornisce un'implementazione di equals
 - Questa versione è etichettata con “final”, in modo da non poter essere sovrascritta nelle sottoclassi
 - Le sottoclassi non contengono ulteriori versioni di equals

Il criterio di confronto **cambia** nelle sottoclassi; oggetti di sottoclassi diverse sono sempre considerati diversi

- Solitamente, questo vuol dire che ciascuna classe controlla che i propri campi siano uguali, nei due oggetti da confrontare
- In questo scenario, il metodo equals va opportunamente **ridefinito in ogni sottoclasse** che abbia un proprio criterio di uguaglianza

- Ad esempio, la seguente specifica rientra nel secondo scenario:
 - **due Employee** sono uguali se hanno lo stesso nome e salario
 - **due Manager** devono avere *anche* lo stesso bonus
 - **un Employee** non è mai uguale ad **un Manager**
- In generale, il metodo equals delle sottoclassi dovrebbe *raffinare* la relazione di uguaglianza, cioè renderla più selettiva, e non viceversa
- Questo principio è coerente con le regole della *progettazione tramite contratti* (Lezione 23)

- Nell'implementazione del secondo scenario, bisogna fare in modo che ciascuna classe si occupi solo dei propri campi, e deleghi alle sue superclassi il confronto dei campi ereditati
- Quindi, il metodo equals nelle sottoclassi (ad es., Manager) dovrebbe preliminarmente invocare quello della superclasse:

```
if (!super.equals(other)) return false;
```

- Poi, se l'oggetto ricevuto come argomento è anch'esso un Manager, dovrebbe procedere ad effettuare i confronti specifici dei Manager

- Quale controllo di tipo dovrebbe fare il metodo equals nella radice della gerarchia?
- E nelle sottoclassi?
- L'obiettivo è confrontare solo oggetti dello **stesso tipo effettivo**
 - in questo scenario, oggetti di tipo effettivo diverso sono sempre considerati diversi
- Non è possibile effettuare questo genere di controllo con l'operatore instanceof
 - perché instanceof tratta allo stesso modo una classe e tutte le sue sottoclassi
- E' necessario usare la classe Class e il metodo **getClass** della classe Object
 - Per un'introduzione a queste funzionalità, si veda la lezione sulla *Riflessione*

- Ad esempio, supponiamo che il metodo `equals(Object o)` della classe `Employee` contenga il seguente test:

```
if (o.getClass() != Employee.class) return false;
```

- Gli oggetti che non sono `Employee` (ad esempio, i `Manager`) sono scartati come differenti
- Tuttavia, questa soluzione impedisce al metodo `equals` di `Manager` di invocare preliminarmente quello di `Employee`, come suggerito in precedenza
- Quindi, è consigliabile utilizzare il seguente test al posto del precedente:

```
if (o.getClass() != getClass()) return false;
```

- In questo modo, sono scartati come differenti soltanto gli oggetti di tipo effettivo diverso, come previsto dallo scenario
- Come ulteriore beneficio, nel metodo `equals` della classe `Manager` non sarà necessario verificare che l'oggetto ricevuto come argomento sia un `Manager`, perché questo test sarà già stato effettuato da `equals` di `Employee`

Ricapitolando, nel caso in cui adottiamo il secondo scenario standard, otteniamo una soluzione di questo tipo:

In Employee:

```
public boolean equals(Object o) {  
    if (o==null)  
        return false;  
    if (o.getClass()!=getClass())  
        return false;  
    Employee e = (Employee) o;  
    // confronta i campi di Employee  
    ...  
}
```

Questo cast è sicuro

Questo controllo
non è ridondante

In Manager:

```
public boolean equals(Object o) {  
    if (!super.equals(o))  
        return false;  
    Manager m = (Manager) o;  
    // confronta i campi di Manager  
    ...  
}
```

Anche questo
cast è sicuro

Valutare la validità delle seguenti specifiche per equals:

	Employee- Employee	Employee-Manager	Manager- Manager	Validità	Scenario
(a)	stesso nome	mai	stesso nome		
(b)	stesso nome	stesso nome	stesso nome		
(c)	stesso nome	mai	stesso nome e stesso bonus		
(d)	stesso nome	stesso nome e bonus 0 per il Manager	stesso nome e stesso bonus		
(e)	stesso nome	stesso nome	stesso nome e stesso bonus		

Valutare la validità delle seguenti specifiche per equals:

	Employee- Employee	Employee-Manager	Manager- Manager	Validità	Scenario
(a)	stesso nome	mai	stesso nome	ok	2?
(b)	stesso nome	stesso nome	stesso nome	ok	1
(c)	stesso nome	mai	stesso nome e stesso bonus	ok	2
(d)	stesso nome	stesso nome e bonus 0 per il Manager	stesso nome e stesso bonus	ok	-
(e)	stesso nome	stesso nome	stesso nome e stesso bonus	non transitiva (perché?)	-

- I due scenari standard **non coprono tutti i casi** possibili
- In particolare, sono esclusi i casi in cui il criterio non è uniforme, e oggetti di classi diverse possono in alcune circostanze essere considerati uguali
- Concretamente, consideriamo la seguente specifica ((d) nella tabella precedente):
 - due Employee sono uguali se hanno lo stesso nome e salario
 - due Manager sono uguali se hanno anche lo stesso bonus
 - un Employee e un Manager sono uguali se hanno lo stesso nome e salario, e il Manager ha il bonus pari a zero
- Sorprendentemente, l'implementazione di questa specifica è **estremamente complessa**
- Almeno, se si vogliono rispettare i seguenti principi:
 - le classi non dovrebbero essere consapevoli delle loro sottoclassi
 - ciascuna classe dovrebbe controllare solo i propri campi
- Quindi, le specifiche di questo tipo sono **sconsigliate**
- Facciamo comunque qualche tentativo di implementazione...

In Employee:

```
public boolean equals(Object o) {  
    if (!(o instanceof Employee))  
        return false;  
  
    Employee e = (Employee) o;  
  
    return (name.equals(e.name) &&  
            salary==e.salary);  
}
```

In Manager:

```
public boolean equals(Object o) {  
    if (!super.equals(o))  
        return false;  
    if (!(o instanceof Manager))  
        return bonus==0;  
  
    Manager m = (Manager) o;  
    return bonus == m.bonus;  
}
```

Non rispetta la specifica e realizza una relazione **non simmetrica**:

se un Manager "m" ha lo stesso nome e salario di un Employee "e", ma ha il bonus diverso da zero, avremo `e.equals(m)==true` mentre `m.equals(e)==false`

La specifica prevede "false", quindi dobbiamo correggere equals di Employee...

Se l'argomento è un Manager, equals di Employee deve controllare "bonus==0":

In Employee:

```
public boolean equals(Object o) {
    if (!(o instanceof Employee))
        return false;

    Employee e = (Employee) o;
    if (!(name.equals(e.name) &&
        salary==e.salary))
        return false;

    if (e.getClass()!=Employee.class)
        return e.isCompatibleWithEmployee();
    else return true;
}

public boolean isCompatibleWithEmployee() {
    return true;
}
```

In Manager:

```
public boolean equals(Object o) {
    if (!super.equals(o))
        return false;
    if (!(o instanceof Manager))
        return bonus==0;

    Manager m = (Manager) o;
    return bonus == m.bonus;
}

public boolean isCompatibleWithEmployee() {
    return bonus==0;
}
```

Di nuovo, **non rispetta la specifica** e realizza una relazione **non riflessiva**:

infatti, `m.equals(m)==false`

Arriviamo a questa implementazione, **corretta**, ma poco pulita e leggibile:

In Employee:

```
public boolean equals(Object o) {
    if (!(o instanceof Employee))
        return false;

    Employee e = (Employee) o;
    if (!(name.equals(e.name) &&
        salary==e.salary))
        return false;
    if (getClass()==Employee.class)
        return e.isCompatibleWithEmployee();
    if (e.getClass()==Employee.class)
        return isCompatibleWithEmployee();
    return true;
}

public boolean isCompatibleWithEmployee() {
    return true;
}
```

In Manager:

```
public boolean equals(Object o) {
    if (!super.equals(o))
        return false;
    if (!(o instanceof Manager))
        return true;

    Manager m = (Manager) o;
    return bonus == m.bonus;
}

public boolean isCompatibleWithEmployee() {
    return bonus==0;
}
```

- Perché non è opportuno specializzare il tipo del parametro di equals?
- In altre parole, perché è meglio effettuare l'**overriding**, piuttosto che l'**overloading**?
- Consideriamo il seguente frammento:

```
Employee e1 = new Employee(...), e2 = new Employee(...);  
Object o1 = e1, o2 = e2;  
o1.equals(o2);  
o1.equals(e2);  
e1.equals(o2);  
e1.equals(e2);
```

- Se la classe Employee effettuasse l'*overloading* di equals, con un metodo del tipo

```
public boolean equals(Employee e)
```

quale versione di equals verrebbe chiamata nei quattro casi qui sopra?

- Perché non è opportuno specializzare il tipo del parametro di equals?
- In altre parole, perché è meglio effettuare l'**overriding**, piuttosto che l'**overloading**?
- Consideriamo il seguente frammento:

```
Employee e1 = new Employee(...), e2 = new Employee(...);  
Object o1 = e1, o2 = e2;  
o1.equals(o2);  
o1.equals(e2);  
e1.equals(o2);  
e1.equals(e2);
```

- Se la classe Employee effettuasse l'*overloading* di equals, con un metodo del tipo
public boolean equals(**Employee** e)
soltanto la quarta invocazione nel frammento qui sopra chiamerebbe il metodo equals di Employee
- Le altre tre invocazioni chiamerebbero quello ereditato dalla classe Object, con risultati inattesi
- Se invece la classe Employee effettuasse l'*overriding* di equals, tutte le quattro invocazioni chiamerebbero il metodo ridefinito in Employee

- Il metodo equals interagisce con diverse funzionalità offerte dalla libreria standard Java
- In particolare:
 - con il metodo hashCode di Object
 - con le collezioni offerte dalla libreria Java Collection Framework
- Queste interazioni saranno trattate nelle lezioni successive

- C#:
 - Consente overloading dell'operatore == (e di molti altri operatori)
 - Consente overriding del metodo Equals(Object) della classe Object
- C++:
 - Consente overloading dell'operatore == (e di tutti gli operatori)
 - Esempio: uguaglianza tra stringhe

```
template<typename _CharT, typename _Traits, typename _Alloc>
inline bool
operator==(const basic_string<_CharT, _Traits, _Alloc>& __lhs,
            const basic_string<_CharT, _Traits, _Alloc>& __rhs)
{ return __lhs.compare(__rhs) == 0; }
```

- Nessuna classe "Object"

- Data la seguente classe.

```
public class Z {  
    private Z other;  
    private int val;  
    ...  
}
```

- Si considerino le seguenti specifiche alternative per il metodo equals
- Due oggetti x e y di tipo Z sono uguali se:
 - a) x.other e y.other puntano allo stesso oggetto ed x.val è maggiore o uguale di y.val
 - b) x.other e y.other puntano allo stesso oggetto ed x.val e y.val sono entrambi pari
 - c) x.other e y.other puntano allo stesso oggetto oppure x.val è uguale a y.val
 - d) x.other e y.other sono entrambi null oppure nessuno dei due è null ed x.other.val è uguale a y.other.val
- Dire quali specifiche sono valide e perché
- Implementare la specifica (d)